Calling `AddHttpLogging()` adds the necessary services for the HTTP logging middleware to the IoC container and customizes the options used by the middleware for what to display. `AddHttpLogging` isn't exposed directly on the `Services` property; it's an extension method that provides a convenient way to encapsulate all the code required to set up HTTP logging. This pattern of encapsulating setup behind extension methods is common in ASP.NET Core.

As well as registering framework-related services, the `Services` property is where you'd register any custom services you have in your application, such as the example `TaxCalculator` discussed previously. The `Services` property is an `IServiceCollection`, which is a list of every known service that your application will need to use. By adding a new service to it, you ensure that whenever a class declares a dependency on your service, the IoC container will know how to provide it.

As well as configuring services, `WebApplicationBuilder` is where you customize other cross-cutting concerns, such as logging. In listing 3.4, I showed how you can add a logging filter to ensure that the logs generated by the `HttpLoggingMiddleware` are written to the console:

```
builder.Logging.AddFilter(
    "Microsoft.AspNetCore.HttpLogging", LogLevel.Information);
```

This line ensures that logs of severity `Information` or greater created in the `Microsoft .AspNetCore.HttpLogging` namespace will be included in the log output.

NOTE I show configuring log filters in code here for convenience, but this isn't the idiomatic approach for configuring filters in ASP.NET Core. Typically, you control which levels are shown by adding values to appsettings.json instead, as shown in the source code accompanying this chapter. You'll learn more about logging and log filtering in chapter 26.